

PCD – Programmazione Concorrente e Distribuita

Assignment 4

Clustered Smart Home Alarm System

Exercise 1

Authors

Alex Santini

Francesco Valentini

Academic Year 2025/2026

16 August 2026

Contents

1	Introduction	1
2	Problem Analysis	2
3	Design	3
3.1	Trade-offs	3
4	Validation	4
5	Conclusions	5

1 Introduction

The assignment asks to distribute the smart home alarm system over an Apache Pekko Cluster while preserving the same alarm behavior as the previous assignment. The system must support at least three nodes, allow sensors and keypads to run on different nodes, and recover safely if the control unit is restarted.

The important point is that distribution must not change the meaning of the alarm system. A sensor activation still has the same effect, the timing rules still apply, and the control unit must remain the only authority for the system state. The cluster is therefore an execution detail, not a change in the domain rules.

2 Problem Analysis

The main concurrency issues are ownership of the alarm state, asynchronous sensor input, and failure recovery.

- The control unit is the single authority for the alarm state machine.
- Sensors and keypads are independent actors that communicate only through messages.
- The control unit must ignore sensor events while recovering, because a restart loses the previous in-memory state.
- Timing rules still matter, so arming and intrusion delays must remain deterministic.

This setup creates a classic distributed-systems problem: the system must stay responsive even when actors live on different nodes, but it must also avoid unsafe assumptions after failures. The recovery state is therefore not an implementation accident. It is the safe default that prevents the restarted node from acting as if it knew the previous alarm condition.

3 Design

The implementation uses typed actors and receptionist-based discovery.

- The control unit registers itself and discovers sirens dynamically.
- Keypads subscribe to control-unit updates and forward PIN submissions plus full or partial arming requests.
- Sensors keep a stable `sensorId` and send serializable `SensorInfo` messages.
- The runtime startup helper builds the Pekko Cluster configuration for three supported nodes.

The non-distributed shape of the previous assignment is preserved wherever it does not conflict with clustering. The project still contains the local `RootActor`, `DemoMain`, timestamped `SensorEvent`, and `AlertDevice` abstraction. The cluster-specific additions are limited to node startup, receptionist discovery, serialization markers, and recovery after control-unit recreation.

The control unit starts in `RECOVERY` after creation or restart. Only a correct PIN can move it back to `DISARMED`. Until then, arming commands and sensor activations are ignored or logged.

The design keeps the code small by separating responsibilities cleanly. Startup code builds the cluster configuration, actors implement the alarm protocol, and shared messages remain serializable and stable across nodes. This makes the system easy to test locally while still matching the distributed deployment model required by the assignment.

3.1 Trade-offs

This solution prefers clarity over maximum generality. It supports the required distributed roles and recovery semantics, but it does not try to add persistence, replication, or more advanced cluster topologies. That choice keeps the code aligned with the assignment and avoids introducing complexity that would not improve the learning goals.

4 Validation

The implementation is covered by unit and integration tests.

- `ControlUnitActorTest` checks the state machine, arming modes, timing behavior, and recovery rules.
- `SensorActorTest` and `KeypadActorTest` check receptionist discovery and message delivery.
- `ClusteredSystemTest` verifies three-node cluster formation, sensor handling, and restart recovery.
- `NodeStartupTest` checks the startup parser and cluster seed-node generation.

The tests are useful not only because they confirm correctness, but because they document the intended distributed behavior. In particular, they lock in the three-node setup, the recovery semantics, and the fact that the control unit never assumes a dangerous default after a restart.

5 Conclusions

The project keeps the alarm semantics from the previous assignment, but makes the deployment distributed and failure-aware. The design is intentionally small: typed messages, a single state owner, dynamic discovery, and safe recovery are enough to satisfy the assignment while keeping the code easy to reason about.